

STRATUM MVO — Development Roadmap

Multi-Vehicle Orchestration — plan of work

Derived from NXM-SW-ARCH-STRATUM-MVO-001 v0.2 · Nexam Systems · Company Proprietary

Plain-English plan for adding **Multi-Vehicle Orchestration** to STRATUM-Desktop, based on **NXM-SW-ARCH-STRATUM-MVO-001 v0.2**. This document does **not** change any code. It says what we are going to build, in what order, and what we need answers on before we start.

1. What we are actually building

Today STRATUM flies one Dagger at a time. The operator picks a target, sets a standoff distance and bearing, and PX4 flies the aircraft to that hold point.

The MVO subsystem lets an operator **coordinate three Daggers onto one target**, each parked at its own bearing on a shared "standoff ring". All three fly independent standoff missions — they don't talk to each other, they don't know each other exists. STRATUM plans the whole picture, staggers the launches, and makes sure their run-in paths don't cross.

Key idea from the spec: we are **not** writing a new autopilot behaviour. PX4 already does standoff. We just teach STRATUM to run three of them at once, on different bearings, at different altitudes.

2. What we already have (don't redo)

The architecture doc did a scan of the code and confirmed that most of the plumbing is already there. This is why the change is smaller than it looks.

Thing we need	State today
Multi-vehicle instantiation	Already live. <code>MultiVehicleManager</code> is stock and unblocked.
Per-vehicle MAVLink command queue with retry	Already per-vehicle. Three concurrent commands = safe.
Per-vehicle links, facts, GPS accuracy	Already per-vehicle. Nothing new to plumb.
Map showing every connected vehicle	Already draws all of them.
PX4 standoff mode + <code>DO_STANDOFF</code> (31010) wire	Unchanged. We reuse it verbatim.
GNSS horizontal accuracy fact	Already exposed on <code>Vehicle.gpsFactGroup.horizontalAccuracy</code> .

What is single-vehicle-bound today (the only things we have to fix): the Nexam Fly-View QML singletons — `StandoffController`, `EngagementController`, the Set Standoff panel, the surveillance circle, every custom `GuidedAction*` button. They all reach for `activeVehicle`. The MVO layer lives beside them, not on top of them.

3. New modules we need to build

Everything below is **additive**. The single-vehicle path stays exactly as it is today.

3.1 C++ side — `src/Orchestration/`

Module	What it does
<code>OrchestrationManager</code>	Top-level object. Owns the mission, the three agents, and the mission-level state machine. Registered as a QML singleton.
<code>StandoffRing</code>	Radius <code>R</code> , height <code>H</code> . Computes the three slot coordinates. Not stored separately from the target — it's always derived.
<code>RingSlot</code>	One slot: bearing <code>θ</code> , transit level, state. There are exactly three, in bijection with agents.
<code>SlotAssignmentsolver</code>	Picks which vehicle gets which slot. Enumerates the 6 permutations, filters out ones that would cross, scores by path length.
<code>TransitPlanner</code>	Assigns transit altitudes (60m/70m/80m by default) and computes the staggered commit schedule so arrivals converge.
<code>FailsafeVerifier</code>	Reads back the D8 parameter block from each vehicle before ARM . Blocks the mission if any value is wrong.
<code>SeparationMonitor</code>	The only thing that looks across all three vehicles at once. Predicts closest approach per pair, raises an advisory only.
<code>VehicleAgent</code> (×3)	One per participating vehicle. Pinned to one <code>Vehicle*</code> for life — never reads <code>activeVehicle</code> . Owns the per-agent state machine.

3.2 QML side — `src/FlyView/Orchestration/`

Module	What it does
<code>OrchestrationWizard.qml</code>	Five-page wizard: Target → Ring → Slots → Assignment → Commit.
<code>RingEditorOverlay.qml</code>	Live map overlay. Drag three slot handles; enforces the minimum angle between slots.
<code>AssignmentReviewPanel.qml</code>	Shows the proposed assignment, transit levels, path lengths, arrival skew. Operator can override.

Module	What it does
<code>OrchestrationStatusStrip.qml</code>	Three horizontal lanes across the top, one per agent, showing state + link + battery.
<code>SeparationAdvisoryOverlay.qml</code>	Warning banner + "HOLD ALL" button. Does not command avoidance manoeuvres.

Entry point is a new action in `FlyViewToolStripActionList.qml` that opens the wizard.

3.3 Things we do **not** build

Listed here so nobody wastes time on them:

- **A ground-side avoidance controller.** Spec explicitly forbids it. Separation is guaranteed by geometry before takeoff, not by chasing metres in flight.
- **Inter-vehicle datalinks.** No V2V. Vehicles are peers only through STRATUM.
- **A firmware change.** V1 rides on the existing 31010 contract untouched.
- **A trajectory generator.** PX4 owns the hold-point geometry.
- **Streaming offboard setpoints.** Never. Would make the radio link an inner control loop.

4. Roadmap — six stages

Stages are ordered by dependency, not by calendar. The technical risk sits in S0–S2; after that it's mostly UI and polish.

Stage 0 — Prove the fleet flies at all

Goal: get three simulated Daggers in the air at once, each with its own MAV system ID, each commandable independently from STRATUM.

- Stand up a multi-instance PX4 SITL with sysids `1`, `2`, `3`.
- Connect all three to STRATUM. Confirm they appear as three vehicles, not one.
- Command each to a different standoff bearing from the existing single-vehicle path, one at a time, to prove the per-vehicle command queues don't tangle.

Exit criterion: three vehicles visible, three independent standoffs commandable from the existing UI.

Why this comes first: the firmware team's own documentation says the custom modes were statically reviewed but never compile-verified in the environment that produced them. We prove they run before we build anything on top.

Stage 1 — Orchestration module skeleton

Goal: the backend can drive three standoffs at once from code (no UI yet).

- Create `src/Orchestration/` with `OrchestrationManager`, `StandoffRing`, `RingSlot`, `VehicleAgent`.
- Wire it into the CMake tree and register `OrchestrationManager` as a QML singleton next to `multiVehicleManager`.

- Implement the agent state machine: `UNASSIGNED → ASSIGNED → PREFLIGHT → LAUNCH_QUEUED → TAKEOFF → CLIMB_TO_LEVEL → COMMIT_QUEUED → STANDOFF_COMMAND → RUN_IN → ON_STATION`.
- Each agent pins one `Vehicle*` at construction and never lets go.

Exit criterion: from a test harness (or QML console), call `orchestrationManager.commit(target, R, H)` and watch three vehicles fly to three bearings on the ring, each on its own transit level.

Stage 2 — The planner

Goal: the assignment and scheduling logic exists, and it's verifiable in a log.

- `SlotAssignmentSolver`: enumerate 6 permutations, filter by cyclic-order-preserving rule, score by total path length, pick the winner.
- `TransitPlanner`: assign per-vehicle transit levels with ≥ 10 m gaps; compute per-agent commit delays so arrivals converge.
- `FailsafeVerifier`: read back the D8 parameter block from each vehicle, block ARM if any value is wrong, name the offending vehicle and parameter.

Exit criterion: log analysis of a SITL run shows every pairwise separation stayed above the required minimum for the whole flight.

D8 parameter block (from spec §3 D8) — these are the values `FailsafeVerifier` gates on:

Parameter	Required value
<code>NAV_DLL_ACT</code>	non-zero (Hold, or RTL with a deconflicted return altitude)
<code>NAV_RCL_ACT</code>	Hold, or <code>COM_RCL_EXCEPT</code> bit 1 set
<code>COM_DLL_EXCEPT</code>	explicit, not defaulted
<code>RTL_RETURN_ALT</code>	per-vehicle, stratified
<code>STDF_SEQ</code>	1 or 3 (approach_with_height = false) — deconfliction-critical
<code>MAV_SYS_ID</code>	unique across the fleet

Stage 3 — Operator wizard and map UI

Goal: an operator can plan and launch a three-vehicle mission end to end.

- Five-page wizard (`OrchestrationWizard.qml`) — Target, Ring, Slots, Assignment, Commit.
- Ring editor overlay on the map, live `θ_min` constraint on slot handles.
- Assignment review panel showing the proposed pairing, transit levels, path lengths, predicted arrival skew.
- Status strip with three lanes across the top of Fly View.

Exit criterion: an operator flies the mission end to end in SITL without touching a command line.

Stage 4 — Separation monitor and degraded modes

Goal: the failure table in the spec §8 passes.

- **SeparationMonitor** : consumes position, velocity, and GPS accuracy per vehicle. Predicts closest approach over a short horizon per pair. Raises advisory + offers HOLD ALL. Never commands a manoeuvre.
- Degrade-to-advisory when GPS accuracy is stale or missing — never substitute an optimistic value.
- Handle each row in the failure matrix (Figure 17 of the spec) — link loss single/all, one-vehicle abort, separation advisory, GNSS degradation.
- Battery endurance display per agent against planned run-in + hold duration (so the operator sees a critical-battery event coming instead of discovering it).

Exit criterion: every row of the fault matrix behaves as the spec says.

Stage 5 — Flight test

Goal: fly a real three-Dagger ring and confirm the prediction was right.

- Three-vehicle ring with a real target.
- Measured minimum separation, arrival skew, and departure-time separation recorded against what STRATUM predicted.

Exit criterion: the prediction matches reality within stated tolerances. The prediction is the deliverable being tested, not the flight.

Stage 6 (deferred, v2) — Arrival-skew trim

Optional. Only worth building if operations say arrival skew has to be tighter than what staggered commit gives us (a few seconds over a 500 m run-in).

- After a vehicle's standoff mode is confirmed (`nav_state == 9`), send `DO_CHANGE_SPEED` (178) to trim its cruise speed.
- **Critical ordering:** speed command must go **after** mode activation, never before. Before activation, `on_activation` calls `reset_cruising_speed()` and the trim is discarded silently.

5. Open questions we need answers on before building

These are the five rulings the spec left open. Answers change what we build; guesses cost rework. Get them settled early — ideally before Stage 2.

#	Question	Default if nobody answers	Impact if the answer changes
1	Common hold height, or one per slot?	Common <code>H</code> for all three	Wire already supports per-slot; UI needs a control
2	Is arrival skew a hard requirement with a number?	Best-effort via staggered commit; report the skew	If yes, Stage 6 moves from optional into Stage 3

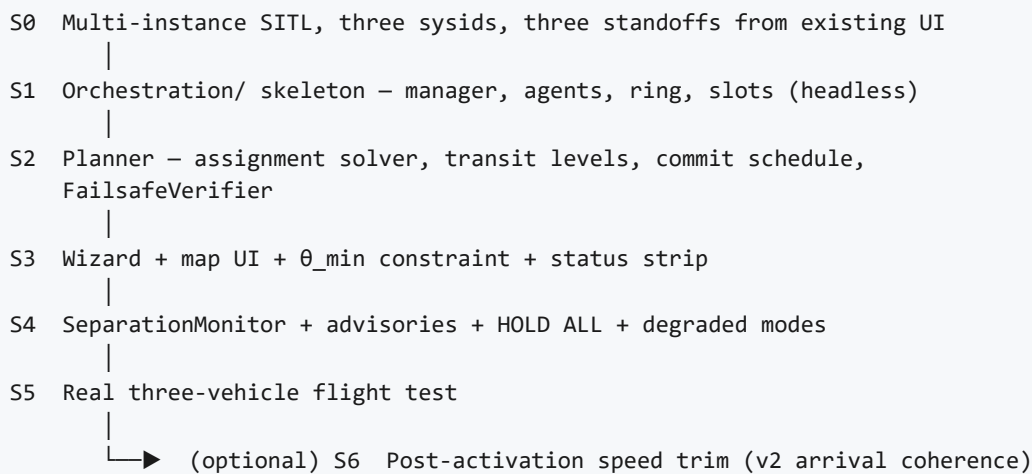
#	Question	Default if nobody answers	Impact if the answer changes
3	Single launch pad, or dispersed launch points?	Single pad, departures separated in time	Dispersed pads make cyclic-order assignment materially more valuable
4	If a vehicle drops mid-mission, does the ring auto-re-solve?	Hold as briefed, offer the re-plan to the operator	Auto-respacing is doable but it's a re-commit under degraded conditions — should be operator-driven
5	Does a coordinated standoff ever become a coordinated engagement?	Excluded from this subsystem entirely	This is the big one. Coordinated engagement is a whole separate safety case — needs its own authorisation chain, abort architecture, and verification. Do not let this arrive by accident.

6. Risks worth naming up front

Not exhaustive — the full register is §10 of the spec. These are the ones that will bite soonest if we don't watch for them.

- **MAV_SYS_ID collision.** Two shipped Daggers with default sysid `1` will present as one vehicle in the UI; the second silently never appears. Looks like a radio fault, isn't one. Belongs in the production build/acceptance procedure, not in code. Stage 0 will surface this the first time we boot two SITL instances with the same id.
- **Stock failsafes are hostile to a ring.** `NAV_DLL_ACT = 0` does nothing on datalink loss; `NAV_RCL_ACT = 2` triggers three uncoordinated RTLs from three points on a ring — the exact crossing pattern we exist to prevent. This is why `FailsafeVerifier` blocks ARM instead of warning. Do not make the check advisory.
- **STDF_SEQ is deconfliction-critical.** Values 2 or 4 fold the climb into the approach and collapse the vertical stratification the whole safety argument depends on. Must be verified as 1 or 3, never assumed.
- **The custom PX4 modes were never compile-verified in the environment that produced them** (per the firmware's own docs). Stage 0's exit criterion is a real SITL flight, not a code review, for exactly this reason.
- **The C2 link becomes mission-critical, not just supervisory.** This is the fundamental trade of ground-side orchestration. It is paid off by the geometry: because separation is guaranteed before the vehicles leave the ground, a link drop degrades the mission but does not endanger the airframes. This only holds if we never build a ground-commanded avoidance controller. Don't build one.

7. Order of work at a glance



Technical risk is front-loaded in S0–S2 (does the firmware actually run three-up, does the assignment logic hold up, does the failsafe verifier catch what it needs to). Once those pass, the rest is UI, monitoring, and flight test.

8. Related documents

- Source spec: [NXM-SW-ARCH-STRATUM-MVO-001.pdf](#) — the full architecture description this roadmap is derived from.
- [STRATUM_Architecture.md](#) — how STRATUM works today. Read this before starting Stage 1 so you know what already exists.
- [STANDOFF_PX4_CONTRACT_PROMPT.md](#) — the wire-level contract with PX4 for the standoff command. Unchanged in v1.